

Depth-First Search

Iterative & Recursive Implementations

C++17 | Graph Traversal | Data Structures

Course	Data Structures and Problem Solving
Assignment	Q50
Student	Youssef Amr Abdelrahman
Student ID	23P0257
University	Ain Shams University
Language	C++17

This report covers the problem statement, solution overview, full C++ source code, complexity analysis, and a head-to-head comparison of the two DFS approaches.

1. Problem Statement

Traverse an **undirected graph** of 11 nodes and 11 edges using the **Depth-First Search (DFS)** algorithm. The graph is represented as an **adjacency list**. The task is to implement DFS in two different ways and handle **disconnected components**:

- **Iterative DFS** — manage traversal using an explicit stack data structure.
- **Recursive DFS** — leverage the system call stack through recursive function calls.

Graph Definition

Edges	{0,1}, {0,4}, {1,2}, {1,3}, {3,5}, {4,6}, {4,7}, {6,8}, {7,9}, {8,10}, {9,10}
Total nodes	11 (labelled 0 – 10)
Graph type	Undirected, unweighted
Representation	Adjacency list

Graph Structure



2. Iterative DFS

The iterative approach replaces the implicit recursion stack with an **explicit stack** (STL `std::stack`). This avoids the risk of stack overflow on very large or deep graphs.

Algorithm Steps

1. Push the starting node onto the stack.
2. Pop a node from the stack. If already visited, skip it.
3. Mark the node as visited and print it.
4. Push all unvisited neighbours onto the stack (in reverse order to preserve left-to-right traversal).
5. Repeat until the stack is empty.
6. Outer loop ensures all disconnected components are covered.

Source Code

```
#include <iostream>

#include <stack>

#include <vector>

using namespace std;

struct Edge { int src, dest; };

class Graph {

public:

    vector<vector<int>> adjList;

    Graph(vector<Edge> const &edges,, int n) {

        adjList.resize(n);

        for (const auto &edge; : edges) {

            adjList[edge.src].push_back(edge.dest);

            adjList[edge.dest].push_back(edge.src);

        }

    }

}
```

```

};

void iterativeDFS(Graph const &graph;, int start, vector<bool> &visited;) {

    stack<int> s;

    s.push(start);

    while (!s.empty()) {

        int vertex = s.top(); s.pop();

        if (visited[vertex]) continue;

        visited[vertex] = true;

        cout << vertex << ' ';

        for (auto it = graph.adjList[vertex].rbegin();

            it != graph.adjList[vertex].rend(); ++it)

            if (!visited[*it]) s.push(*it);

    }

}

int main() {

    vector<Edge> edges = {

        {0,1},{0,4},{1,2},{1,3},{3,5},

        {4,6},{4,7},{6,8},{7,9},{8,10},{9,10}};

    int n = 11;

    Graph graph(edges, n);

    vector<bool> visited(n, false);

    for (int i = 0; i < n; ++i)

        if (!visited[i]) iterativeDFS(graph, i, visited);

    return 0;
}

```

```
}
```

Output

```
0 1 2 3 5 4 6 8 10 9 7
```

3. Recursive DFS

The recursive approach uses the **system call stack** automatically. The code is significantly simpler and more readable, making it the preferred choice for small to medium-sized graphs.

Algorithm Steps

1. Mark the current node as visited and print it.
2. For each unvisited neighbour, call recursiveDFS on that neighbour.
3. Recursion unwinds automatically when all neighbours are visited.
4. Outer loop ensures disconnected components are handled.

Source Code

```
#include <iostream>

#include <vector>

using namespace std;

struct Edge { int src, dest; };

class Graph {

public:

    vector<vector<int>> adjList;

    Graph(vector<Edge> const &edges,, int n) {

        adjList.resize(n);

        for (const auto &edge; : edges) {

            adjList[edge.src].push_back(edge.dest);

            adjList[edge.dest].push_back(edge.src);

        }

    }

};

void recursiveDFS(Graph const &graph,, int vertex, vector<bool> &visited;) {
```

```

        visited[vertex] = true;

        cout << vertex << ' ';

        for (int neighbor : graph.adjList[vertex])

            if (!visited[neighbor])

                recursiveDFS(graph, neighbor, visited);

    }

int main() {

    vector<Edge> edges = {

        {0,1},{0,4},{1,2},{1,3},{3,5},

        {4,6},{4,7},{6,8},{7,9},{8,10},{9,10}};

    int n = 11;

    Graph graph(edges, n);

    vector<bool> visited(n, false);

    for (int i = 0; i < n; ++i)

        if (!visited[i]) recursiveDFS(graph, i, visited);

    return 0;

}

```

Output

```
0 1 2 3 5 4 6 8 10 9 7
```

4. Complexity Analysis

Time Complexity — $O(V + E)$

Both implementations visit every vertex exactly once and process every edge exactly once, giving $O(V + E)$ time where V is the number of vertices and E is the number of edges.

Space Complexity

Iterative	$O(V)$ — visited array + explicit stack (at most V entries)
Recursive	$O(V)$ — visited array + recursion call stack depth up to $O(V)$

5. Iterative vs Recursive — Comparison

Feature	Iterative DFS	Recursive DFS
Stack used	Explicit stack	System call stack
Code complexity	Moderate	Simple & clean
Stack overflow	No risk	Risk on large graphs
Time complexity	$O(V + E)$	$O(V + E)$
Space complexity	$O(V)$	$O(V)$ + recursion stack
Best for	Large graphs	Small / medium graphs

Key Takeaways

- Both approaches produce **identical traversal output**: 0 1 2 3 5 4 6 8 10 9 7.
- The recursive version is **cleaner and easier to read** but risks stack overflow on very large graphs.
- The iterative version is **safer for production use** on large or deep graphs.
- Both correctly handle **disconnected components** via the outer for-loop in main.